CMPE257
Final Report

Group 7

Anoushka Sawant - 015154074
David Wang - 012087348
Hoai An Nguyen - 014719549
Itzel Xu - 017549558
Khush Naidu - 015798328
Sergio Talavera - 017557527

Date: Nov 22, 2024

1. Idea **3**
1.1. Description/Problem Statement 3
1.2. Goals/Objectives 4
2. Work Developed **5**
2.1. Adopted ML algorithms/techniques 5
2.1.1 Dataset 5
2.2.2 Preprocessing steps 5
2.2.3 Models 6
2.2. Performance metrics 9
2.3. Testing Results 10
2.4. Evaluation 13
2.4.1 Pretrained models 13
2.4.2 Model from scratch 15
2.5. Future Work 16
3. Project Management 17
3.1. Final Schedule & Task Distribution (Gantt chart) 17
3.1.1. Task Distribution (Gantt Chart) 17
3.1.2. Project Milestones 18
3.2. Challenges 19
3.3. Learning Outcomes 21
References **22**

1. Idea

1.1. Description/Problem Statement

Deepfake technology, which uses artificial intelligence to create hyper-realistic but fake videos and images, has become a significant concern in recent years. The ability to manipulate visual and audio content so convincingly poses numerous threats across various domains, including politics, personal privacy, and cybersecurity.

A notable incident highlighting the severe impact of deepfakes occurred in South Korea in August 2024. This crisis involved the creation and distribution of deepfake pornography targeting students and teachers across various schools and universities. This scandal affected nearly 200 individuals, forcing many to change their lives drastically due to the humiliation and privacy violations they experienced. [1]

Another incident that showcases the financial impact of deepfakes happened in the UK in September 2019. The CEO of a UK-based energy company believed they were on a phone call from their boss, who has a parent company in Germany. The scammers used AI voice technology to impersonate their boss to get the UK CEO to transfer €220,000 to a fake Hungarian supplier. This incident marks one of the first cases of AI voice software used to commit fraud. [2]

These incidents underscore the urgent need for robust deepfake detection technologies and stricter regulations to prevent such abuses in the future. To address this issue, it is crucial to develop tools for detecting AI-generated human faces and voices.

1.2. Goals/Objectives

The project will study and employ other AI methods besides using GANs, which are intended to both create and recognize deep fakes. Because of this, the detection system will be able to learn the most minute differences and features, i.e. it can be generalized in different cases. In addition, we will use transformer and neural networks to make the model more accurate and robust in solving limitations like frame to frame variations observed in deep fake videos.

Our aim after all is to design and implement a system which is capable of agnostic and deep fakes burst detection.

At its core, this project revolves around building a successful deep fake detection system that can tackle the issue we have highlighted. We want to investigate the potential of generative adversarial networks (CycleGAN), transformers, neural networks, and other techniques to develop a model that can accurately identify deep-faked videos. We plan to use the DFDC dataset. The top-performing model on the public dataset achieved an average precision of 82.56% [3]. Our goal is to surpass this score. Furthermore, we intend to build on current industry standard models by reviewing newer research papers and incorporating their findings into our model, with the goal of making one of the best tools in the industry.

2. Work Developed

2.1. Adopted ML algorithms/techniques

2.1.1 Dataset

The dataset we are using in this project is Meta's Deepfake Detection Challenge Dataset which is 500GB, totaling around 100,000 videos. It consists of real videos (unmodified, authentic videos), fake videos (deepfakes of said real videos), "original" videos (videos without

deepfakes), as well as dimensional metadata for each video. We decided to use a small subset of the entire dataset to begin model training. The subset was 5GB in size, totalling to around 800 videos. [3]

2.2.2 Preprocessing steps

1. Capture frames from each video - extract images from video files and load them into the dataset.
    - Capturing frames increased our dataset greatly. After our first round of frame capturing, the dataset increased to over 30GB of frames.
2. Data Cleaning - getting rid of any anomalies, low quality data should be filtered and it should be easy to integrate into the model.
3. Identify faces and crop isolated face images using the face_recognition package to extract the relevant features/landmarks.
    - Face Extraction: isolating faces from video frames
    - Face Alignment: key facial landmarks (e.g., eyes, nose, mouth) are in a standard position
4. Apply data augmentation to make the dataset more balanced and improve model robustness.
5. Balance the data using Underbalancing Technique

Fig. 1 Preprocessing process

2.2.3 Models

There are multiple models we worked on:

1. Using pretrained model with following architecture:
    - 2 InceptionLayer - applied to increase the efficiency and performance of convolutional neural networks. deploy multiple convolutions with multiple filters and pooling layers simultaneously in parallel within the same layer (inception

layer). Intended to let the neural network learn the best weights when training the network and automatically select the more useful features.

- 2 Conv2D layers
- Batch Normalization - applied after each convolutional layer to enhance convergence speed and reduce overfitting.
- Multiscale pooling using MaxPooling2D - to capturing information at various spatial scales and enhance feature extraction
- 2 Dropout layers to mitigate overfitting
- Fully-connected output layer with 1 unit and sigmoid activation

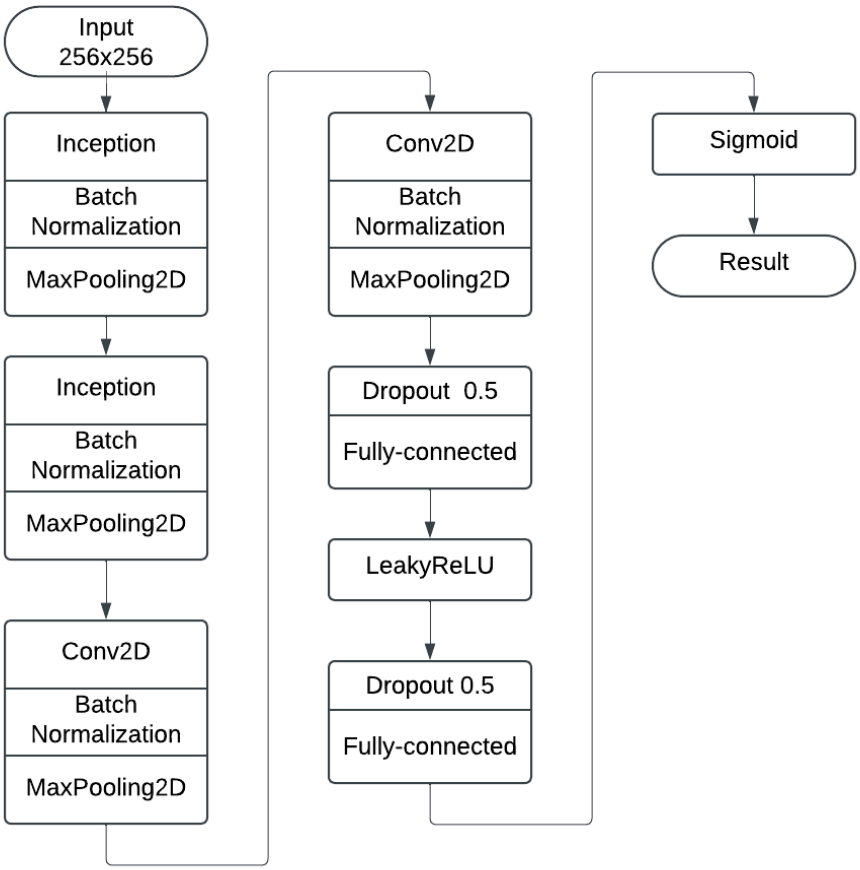

Fig. 2 Pretrained Model Network Architecture

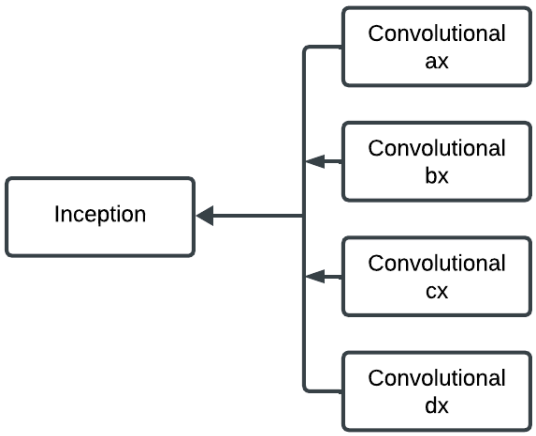

Fig. 3 Inception Layer Architecture

2. Model from scratch

The model we develop closely follows Meso4 architecture [4] with some minor changes.

Model Architecture:

- 4 convolutional blocks composed of:
- Conv2D layers with increasing filters (8->16->32->64) to catch more complexities as the model learns deeper
- L2 regularization to prevent overfitting
- Batch normalization to stabilize learning
- ReLU activation to introduce non-linearity
- Max Pooling to retain essential features
- Dense 256
- Dropout 0.5 to reduce overfitting from train set
- Dense (1) decision layer

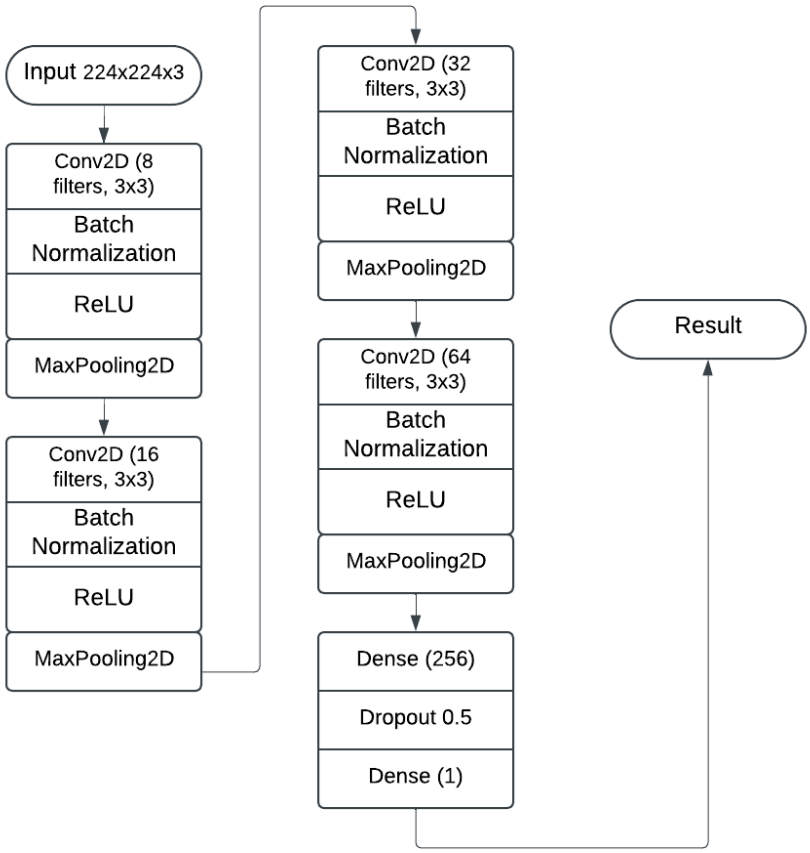

Fig. 4 Model Architecture

2.2. Performance metrics

To evaluate the performance of our model we used the following metrics:

- Accuracy - percentage of correctly classified test images
- Precision - percentage of test images that the CNN predicts as a particular class and that are actually of that class
- Recall - percentage of test images that are of a particular class and that the CNN predicts as that class
- F1 score - harmonic mean of precision and recall
- Loss function - measure how well the CNN is performing on the training data. It's calculated as the difference between the predicted labels and the actual labels of the training images.

2.3. Testing Results

Testing results for pretrained models:

- Accuracy - 0.674
- Precision - 0.674
- Recall - 1.0
- F1 score - 0.805
- Inference time - 21 frames/sec
- The log loss function results are as follows:
    - random/1/0/0.5 loss
    - - 0.715/11.728/24.315/0.693
    - Our loss - 0.632

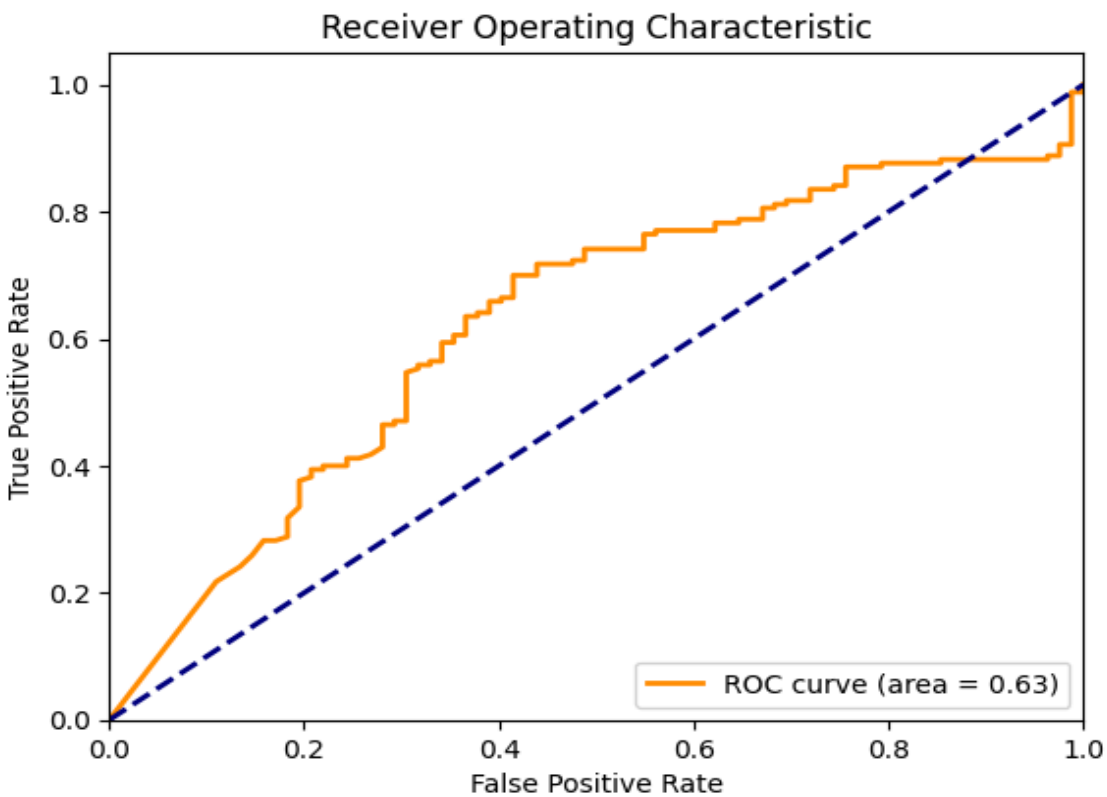

Fig. 5 ROC for best pretrained model

The high recall (1.0) with a moderately good precision and accuracy (0.674), and F1 score (0.805) suggests that the model is

very good at detecting positive instances (fake videos), but may have some false positives. This indicates that pretrained models might be suitable for applications where it is critical to identify all positive cases, even if it means having some false positives, such as fraud detection or medical screening.

Testing results for our model:

- Accuracy - 0.84
- Precision - 0.82
- Recall - 1.0
- F1 score - 0.90

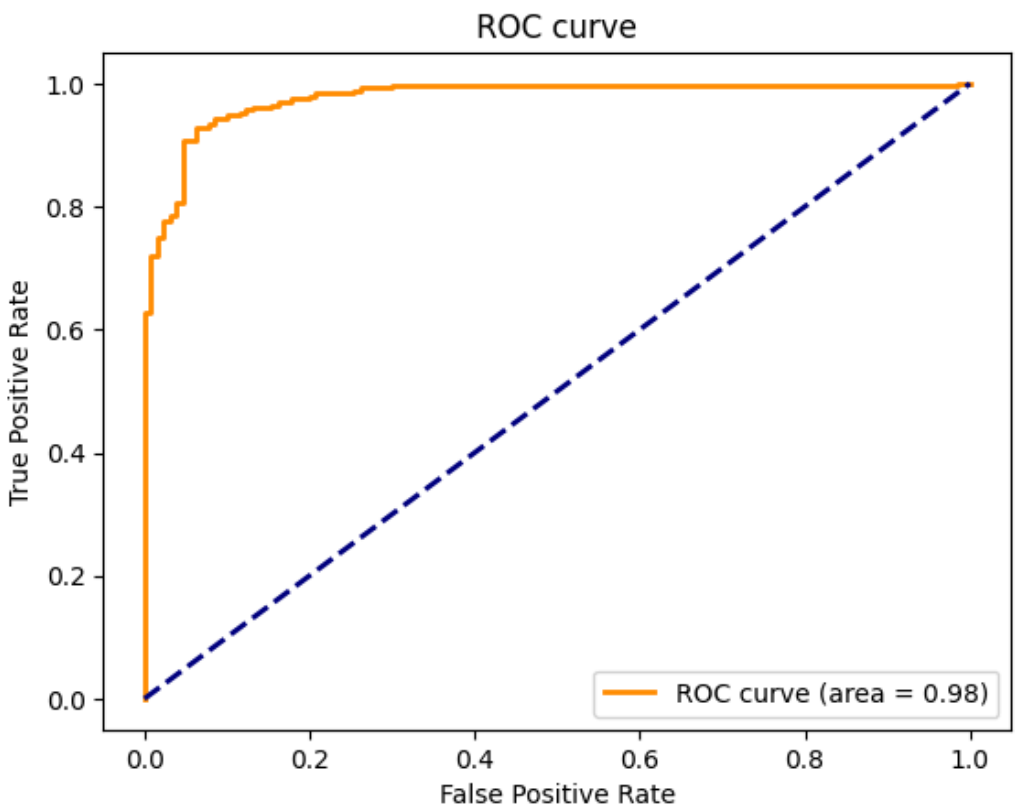

Fig. 6 Roc Curve for our model

The model developed from scratch has better performance with high recall (1.0), precision (0.82), accuracy (0.84), and F1 score (0.90).

The log loss function results with different epochs are as follows:

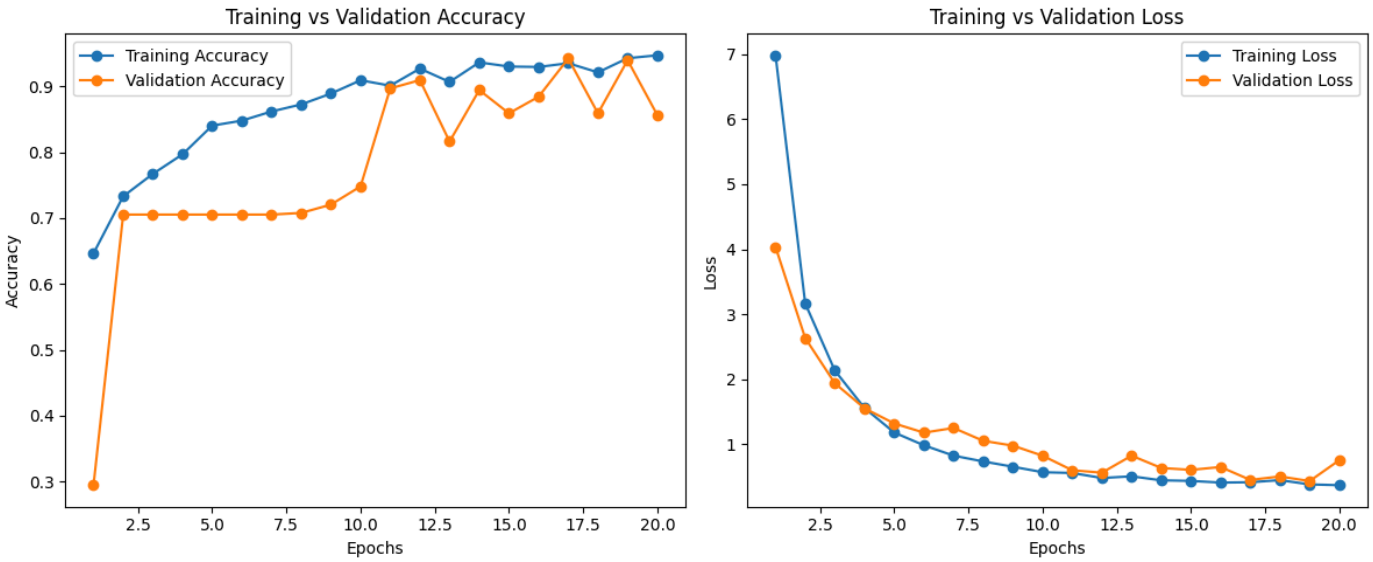

Fig. 7 Training vs Validation for Accuracy and Loss

This model performs best around the range of 11-19 epochs. Performance peaked at epoch 17 for our final run, with a train accuracy of 92.8% and test accuracy of 94.2%.

2.4. Evaluation

2.4.1 Pretrained models

Issues we've faced are high validation loss that leads to lower validation accuracy: The pretrained model's validation loss remains high, while validation accuracy stays low, indicating overfitting. This is most likely due to the pretrained model's complexity makes it prone to overfitting with the current dataset.

Some observations we made with this model:

- High Validation Loss and Low Validation Accuracy: The validation loss is extremely high, starting at 42.4733 and only reducing to 13.4836, while the validation accuracy remains low at 0.2772

throughout the epochs. The model is overfitting to the training data, failing to generalize well to the validation set. This could be due to the pretrained model being insufficiently adapted to the specific characteristics of the new data.

- Learning Rate Adjustments: The learning rate starts at 1.0000e-04 and decreases across epochs. The large initial learning rate might have caused instability in learning. The initial learning rate might have been too high, causing large and destabilizing updates to the model weights. Subsequent reductions may not have been sufficient to stabilize training.

- Validation Performance: Pretrained model validation loss is 4.4565, and validation accuracy is 0.6931. After fine-tuning, initial validation loss is 1.5182 and increases to 2.7387. Validation accuracy starts at 0.6980 and slightly improves to 0.7228. The model is overfitting to the training data, as indicated by the increasing validation loss despite the slight improvement in validation accuracy.

- Training vs. Validation Loss Discrepancy: In DFDC training loss decreases from 1.4782 to 0.8756, whereas validation loss increases. In F2F training loss decreases from 1.4782 to 0.8756, while validation loss increases from 1.5182 to 2.7387. The model is fitting the training data well but failing to generalize to the validation set, suggesting overfitting.

Potential causes include model complexity, learning rate and other hyperparameters might not be appropriately set and data distribution mismatch. The pretrained models we used have complex architecture which requires careful tuning of the hyperparameters. It also could be not suitable for the dataset we are using, causing overfitting. There might be a significant difference between the distribution of the pretrained model's original data and our dataset.

We propose multiple improvements for using those pretrained models:

- Regularization Techniques:
  - Dropout: Introduce dropout layers to reduce overfitting.
  - L2 Regularization: Apply L2 regularization to penalize large weights.
- Learning Rate Adjustment:

    - Start Lower: Use a lower initial learning rate for fine-tuning (e.g., 1e-5) to ensure more stable learning.
    - Adaptive Learning Rates: Implement learning rate schedulers like ReduceLROnPlateau to adjust the learning rate based on performance metrics.
- Layer Freezing Strategy:
    - Freeze Early Layers: Initially freeze the early layers of the pretrained model and only fine-tune the later layers to adapt more effectively to the new data.
- Cross-Validation: Employ cross-validation to ensure that the model generalizes well across different subsets of the data.

2.4.2 Model from scratch

The model trained from scratch with added L2 regularization proved to be an effective solution to the overfitting issues encountered during the initial training process. It is this regularization that made it possible for the model not to concentrate on the training data too much and this improved the overall performance of the model. One other additional benefit from training the model from scratch was that the model focused more on the requisite feature relationships without the distraction of any pretrained biases hence a more fine grained understanding of the data set. Furthermore, training models from scratch helped strengthen relationships between important features without the influence of pretrained weights.

The model was able to achieve this level of accuracy without over-fitting or under-fitting and so seems to be fairly well balanced, but it does still require improvement in certain areas. One aspect that can be targeted for enhancement is the learning rate. The current learning rate of 0.001 would also best be dynamically lowered from 0.1 in order to allow for a more gradual approach to updates to be taken, which would improve the likelihood of convergence occurring in a stable way. In addition to this, cross-validation should also be used in order to confirm that the performance of the model is consistent across different data splits using the same policy, this would further confirm the strength and generalizability of the model.

With a general accuracy of 84%, we can assume that model is not overfit or underfit but can certainly perform better

In the end, the developed model outperformed the pretrained models we tested and proved that it is able to learn on its own by getting impressive results without the usage of any pretrained weight. This accomplishment speaks well of the model architecture and the well-set hyperparameters during the training process - the set objective to have high accuracy on the DFDC dataset has also been achieved.

2.5. Future Work

In the future, we can expand our dataset to include videos generated from GANs (Generative Adversarial Networks) or new deepfake manipulation techniques to diversify training samples. Alternatively, we can use other publicly available datasets such as FaceForensics++. We can also implement different data augmentation techniques specific to deepfake artifacts (e.g., random noise to images to simulate artifacts or imperfections, distortions such as stretching, scaling, or shifting, and blurring of some areas of the images) to improve the model's robustness to subtle manipulations. When it comes to preprocessing steps, we can add an additional step to incorporate padding into the frames with the face extracted.

To improve the performance of the models discussed, we can experiment with different learning rates, further fine-tuning the parameters and layers to optimize models for better performance. To help the models better capture artifacts specific to deepfakes without requiring extra layers, we can explore various kernel sizes and filter numbers. We can adjust the batch size and dropout rates for hyperparameter optimization to enhance the generalization. Another configuration worth exploring is residual connections to improve gradient flow and reduce the risk of vanishing gradients. On top of that, we can do more research on new deepfake generators and consider other artifacts left by those new models and ways to combat them.

The model we developed can be applied to different fields requiring authentication and verification. It can be used in media sources to check the content for false information in news, especially during political campaigns or major global events. When incorporated into social media platforms and personal devices, it could help with content moderation to remove harmful deepfake video content before it reaches the public. It can also protect individuals from maliciously altered videos used for blackmail, harassment, or defamation. To prevent cases like the one we covered in Korea, where teenagers and young adults were blackmailed with deepfakes of a sexual nature created by their schoolmates [1].

This model can prevent fraud if integrated into existing systems to detect deepfake-based phishing attacks or impersonations in video calls and messages. The model can also be used in legal and forensics to check court-submitted evidence for authenticity. Additionally, it could increase public awareness by demonstrating how common deepfakes are and how seamlessly they blend into the content we consume on an everyday basis.

3. Project Management

3.1. Final Schedule & Task Distribution (Gantt chart)

3.1.1. Task Distribution (Gantt Chart)

PROJECT DATA				GANTT SCHEDULE															
TASK	Team Member	START DATE	COMPLETION DATE	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12	Week 13	Week 14	Week 15	Week 16
Sprint 0 (Inital Research)																			
Project Brainstorm	Everyone	09/10/24	09/24/24																
Individual project presentation	Everyone	09/17/24	09/24/24																
Finalizing Project Goal	Everyone	09/17/24	09/24/24																
Initial project report	Everyone	09/24/24	10/08/24																
Gantt Chart	Everyone	09/24/24	10/08/24																
Reviewing and compiling datasets	Sergio, Anoushka	10/01/24	10/15/24																
Conduct literature review on deep fake detection techniques	Itzel, An	10/01/24	10/15/24																
Identify tools and technologies to be used	Itzel, Anoushka	10/01/24	10/15/24																
Sprint 1 (Preprocessing & Model Architecture)																			
Research and select pretrained models	Itzel, An	10/08/24	10/15/24																
Study Meso4 Architecture	Khush	10/08/24	10/15/24																
Data cleaning and preprocessing (face extraction, normalization)	Sergio, Anoushka	10/08/24	11/05/24																
Group Presentation	Everyone	10/08/24	10/10/24																
Develop the model architecture	Khush, Itzel	10/15/24	10/22/24																
Sprint 2 (Build Initial Model)																			
Model implementation (using TensorFlow/PyTorch)	Khush, Itzel	10/22/24	11/05/24																
Improve preprocessing(data augmentation & balancing)	Sergio, Anoushka	10/29/24	11/05/24																
Evaluate model performance	Khush, Itzel	10/29/24	11/05/24																
Comparing performance with initial model	David, An	11/05/24	11/19/24																
Case Study	Everyone	11/05/24	11/12/24																
Sprint 3 (Model Evaluation & Documentation)																			
Pre-trained model fine-tuning	Itzel	11/12/24	11/21/24																
Improve Model from scratch	Khush	11/12/24	11/21/24																
Visualization and Performance metrics	David, An	11/19/24	11/21/24																
Final report detailing our findings and future works	David, An	11/19/24	11/22/24																
Final Presentation	Everyone	11/19/24	11/21/24																

Fig. 8 Gantt Chart

3.1.2. Project Milestones

- Have research references regarding current DeepFake detection models methods
  - Have reviewed and researched Machine Learning Algorithms and Models
    - Assigned to: Itzel Xu, Hoai An Nguyen
  - Understand current challenges and where to improve in existing DeepFake detection models
    - Assigned to: Sergio Talavera, Anoushka Sawant
- Data Collection and Preprocessing
  - Ensure that data is clean and conduct analysis to identify and understand how to utilize the data
    - Assigned to: Sergio Talavera, Anoushka Sawant
- Model Selection/Initial Model Development
  - Have a specific model selected for initial experimentation
    - CNN, RNN, LSTM, Transformers, etc.
    - Assigned to: Itzel Xu, Khush Naidu, Anoushka Sawant
  - Have a documented baseline performance as a reference for future improvements

        - Assigned to: Hoai An Nguyen
- Model Optimization
    - Begin experimenting with hyperparameter tuning and optimize models based on consistent performance
        - Assigned to: Itzel Xu, Khush Naidu
    - Visualization and evaluation of the model's performance and our model's performance
        - Assigned to: Hoai An Nguyen, David Wang
- Final Report and Presentation
    - Have our findings compiled and organized for classroom presentation and project submission.
        - Assigned to: Hoai An Nguyen, David Wang

3.2. Challenges

Our team chose a complex project because we believe it's essential to research and develop tools to combat the rapid spread of deepfakes. However, some team members were not familiar with advanced deep learning techniques, so we encountered several challenges while working on building our deepfake detection tool:

1. Learning Curve:
   The team encountered a big learning curve as we navigated through complex concepts and the implementation of machine learning algorithms. While all team members are familiar with basic machine learning concepts learned in class, more advanced topics, such as convolutional neural networks (CNNs), application of pre-trained models, integrating modern-day conventional practices like Bath Normalization and Dropout, new activation functions, were challenging for those new to the field.

2. Data Set Size:
   The size of the dataset is 500GB, which far exceeds Google's limit of 100. We also had to generate and store

images associated with the video input. Because of that, we had to split the original dataset and use a smaller subset of videos for model training and testing.

3. Limited Resources:
   Our team's machines lack the processing power to train the model within a reasonable timeframe. This led to a slower experimental process, making it difficult to test different configurations for model optimization.

4. Data Imbalance:
   The dataset we use is highly unbalanced, with significantly more fake videos than real videos. This led to model overfitting since the model tends to memorize larger classes resulting in poor generalization on unseen deepfake data.

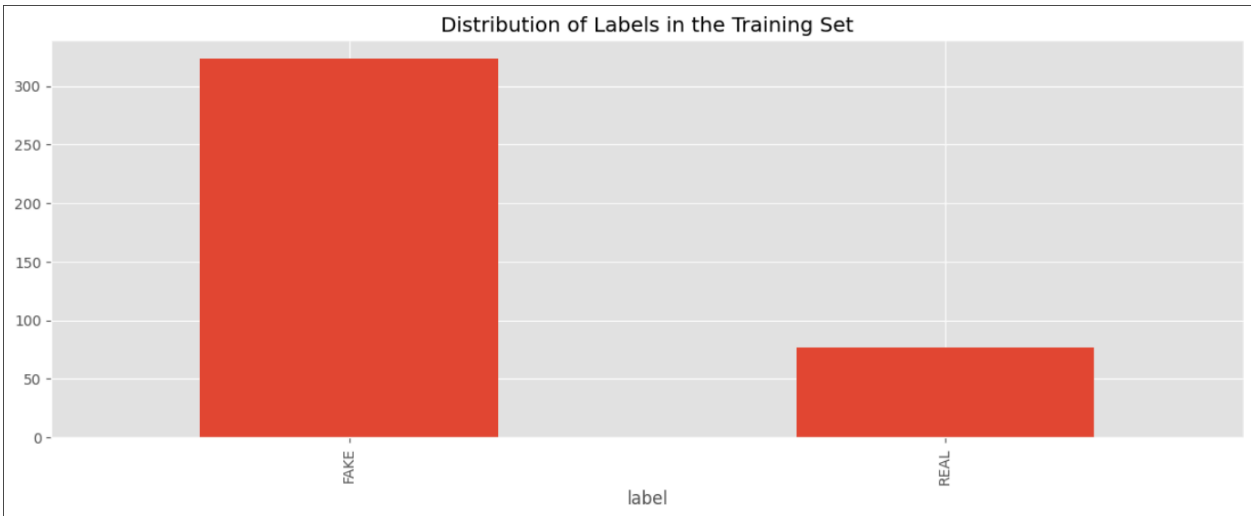

Fig. 9. Distribution of 'Fake' vs 'Real' in the training dataset

Despite the challenges the team faced, we gained valuable insights into deepfake detection techniques and machine learning practices and have hands-on experience developing an ML tool that can make a positive impact.

3.3. Learning Outcomes

We gained a comprehensive understanding of deepfake technologies, focusing on the detection of synthetic videos while addressing key challenges in identifying manipulated content. We also got hands-on experience preparing and preprocessing our dataset videos for deepfake analysis to ensure a robust data foundation. We implemented convolutional neural network (CNN) models for deep fake detection, utilizing pre-trained architectures and developed our own model inspired by Meso4's architecture. In the process, our team deepened its knowledge of advanced CNN features, including multi-scale pooling, dropout, batch normalization, and inception layers, and observed how these factors affect model performance. Additionally, we were able to apply evaluation metrics such as accuracy, precision, F1-score, and ROC to help us analyze the model's performance. This project helped the team with gaining practical skills and a deeper understanding of state-of-the-art methods in this critical area of machine learning.

References

[1] J. Mackenzie and L. Choi, “South Korea: The deepfake crisis engulfing hundreds of schools,” *Bbc.com*, Sep. 03, 2024. https://www.bbc.com/news/articles/cpdlpj9zn9go

[2] J. Damiani, “A Voice Deepfake Was Used To Scam A CEO Out Of $243,000,” Forbes, Sep. 03, 2019. https://www.forbes.com/sites/jessedamiani/2019/09/03/a-voice-deepfake-was-used-to-scam-a-ceo-out-of-243000/

[3] meta, “Deepfake Detection Challenge Dataset,” ai.meta.com, Jun. 25, 2020. https://ai.meta.com/datasets/dfdc/

[4] Afchar, D., Nozick, V., Yamagishi, J. and Echizen, I., 2018, December. Mesonet: a compact facial video forgery detection network. In 2018 IEEE international workshop on information forensics and security (WIFS) (pp. 1-7). IEEE.